

Сортировки на видеокартах

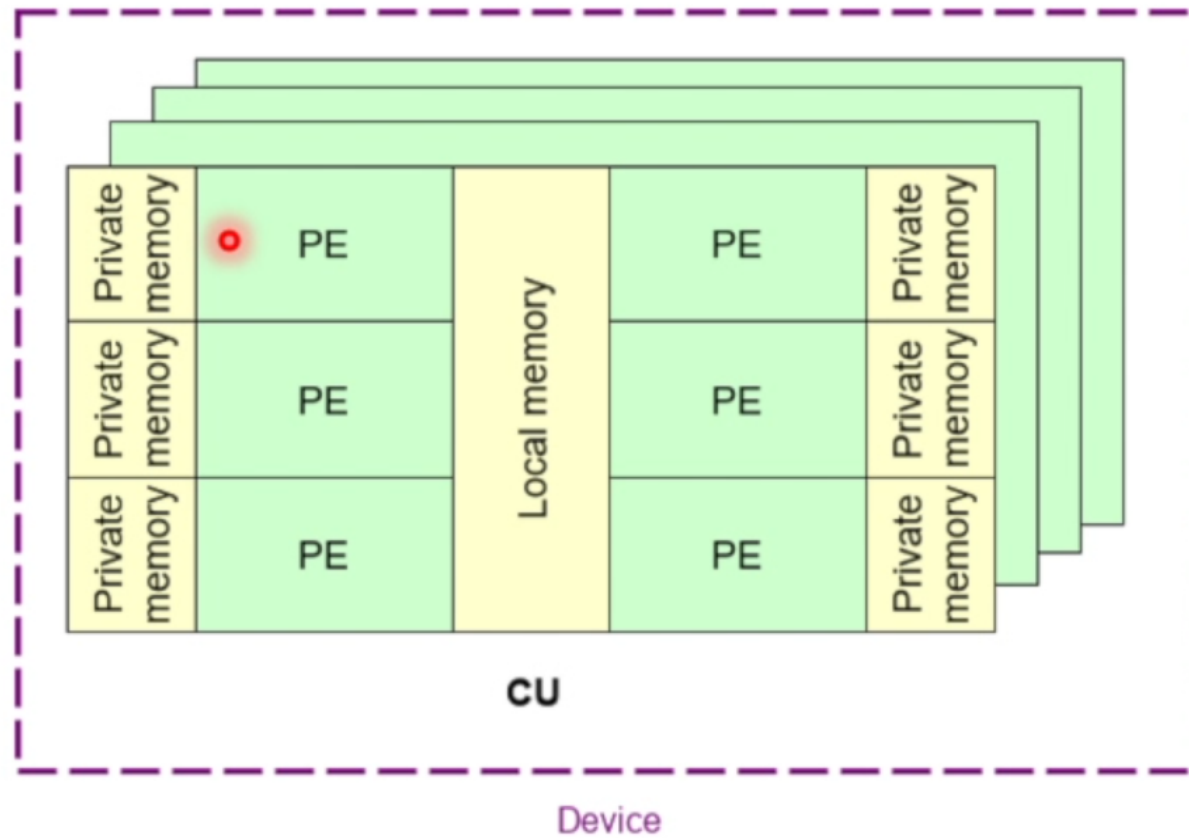
Себелев Максим, DeepSeek

15 мая 2026 г.

Введение

Поведаю вам про сортировку на гри. Все, сразу к делу

Модель памяти OpenCL



Битонная сортировка

Битонная сортировка (Bitonic Sort) — это алгоритм сортировки, который крайне хорошо поддается распараллеливанию. Он имеет асимптотику хуже, чем стандартные алгоритмы сортировки $O(N \log^2 N)$ против $O(N \log N)$, однако при достаточно больших массивах данных (порядка 10^6 элементов) и грамотном использовании на видеокартах может давать куда лучшие результаты, чем стандартные алгоритмы

Определение:

Битонная последовательность - это конечная последовательность $\{a_n\}_{n=0}^N$, для которой существует k : $\{a_n\}_{n=0}^k$ является возрастающей последовательностью, а $\{a_n\}_{n=k+1}^N$ - убывающей (или наоборот). То есть, последовательность является противоположно монотонной слева и справа от своего экстремума.

Для обобщения будем считать монотонные последовательности и последовательности из 1 элемента так же битонными.

Алгоритм реальный

Небольшая поправка - алгоритм определен только для массивов с размером равным степени 2. Однако это не проблема, так как засылая данные на видеокарту мы можем добавить элементов до степени 2 элементов. Добавлять будем элементы максимальные по значения для своего типа (например, для `int` - `INT_MAX`), чтобы после сортировки они остались последними в массиве. В таком случае мы можем считать с видеокарты первые n элементов отсортированного массива, которые будут соответствовать исходному отсортированному массиву.

Поэтому далее смело рассматриваемым только массивы с длиной равной степени 2.

В алгоритме есть 2 понятия: **step** и **stage**

Алгоритм состоит из последовательности step-ов, на каждом из которых выполняется определенная последовательность stage-й.

step: На i -ом step-е последовательно выполняем stage-и от i -го до 0-го.

stage: На i -ом stage делим массив на последовательные боксы по 2^{i+1} элементов (именно из-за этого шага длина массива должна быть равна степени 2). Элементы внутри этого бокса пронумеруем от 0 до 2^{i+1} . Элементы с разницей индексов равной 2^i выстраиваем их в отсортированном порядке. Именно этот шаг и поддается распараллеливанию, потому что все элементы делятся по парам и этим шагам сортировки внутри бокса выполняются для таких пар независимо и так же для всех боксов эти вычисления проводятся независимо.

20 22 2 19 1 16 9 0 12 24 18 8 16 4 24 29 4 5 24 0 15 20 16 9 15 2 17 32 8 11 28 19

step 0.

stage 0: (0, 1) (2, 3) (4, 5)

20 22 19 2 1 16 9 0 12 24 18 8 4 16 29 24 4 5 24 0 15 20 16 9 2 15 32 17 8 11 28 19

step 1.

stage 1: (0, 2) (1, 3) (4, 6) (5, 7) (8, 10)

stage 0: (0, 1) (2, 3) (4, 5)

2 19 20 22 16 9 1 0 8 12 18 24 29 24 16 4 0 4 5 24 20 16 15 9 2 15 17 32 28 19 11 8

step 2.

stage 2: (0, 4) (1, 5) (2, 6) (3, 7) (8, 12)

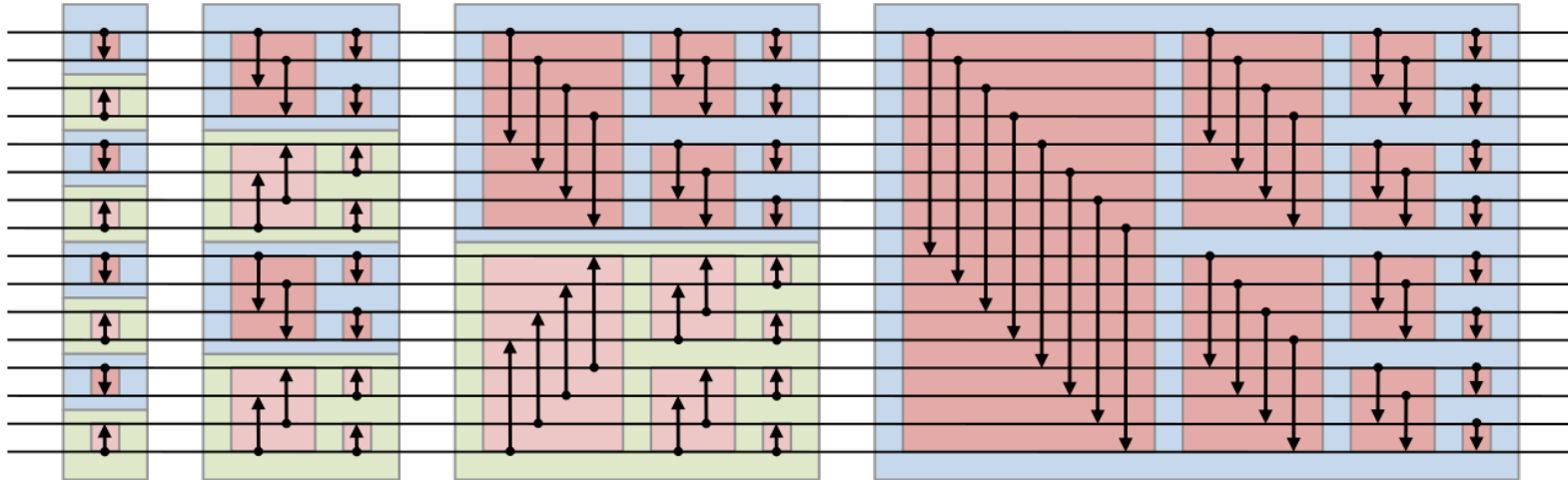
stage 1: (0, 2) (1, 3) (4, 6) (5, 7) (8, 10)

stage 0: (0, 1) (2, 3) (4, 5)

0 1 2 9 16 19 20 22 29 24 24 18 16 12 8 4 0 4 5 9 15 16 20 24 32 28 19 17 15 11 8 2

А зачем было определение? И где битонность то?

Алгоритм, описанный выше на самом деле крайне незаметно создает битонные последовательности и так же незаметно сливает и сплитит x : Действительно, после i -го step-а массив делится на $i+1$ битонных последовательностей. А на следующем step-е соседние последовательности (0 и 1, 2 и 3, 4 и 5, ...) сливаются в одну большую. Собственно здесь замаскировано простым алгоритмом, что серьезные ученые называют bitonic-split и bitonic-merge, отсюда и название. Однако заумные слова не помогают написать эффектный код на OpenCL, поэтому не будем заострять внимание на этом.



Реализация

Первые шаги

Давайте заметим, что размер локальной памяти для CU обычно довольно маленький. Значит не все боксы мы сможем туда помещать. Поэтому в локальной памяти мы сможем работать только на небольших stage-ах. На моем устройстве размера локальной памяти CU хватит лишь на 512 4-байтных int. Теперь давайте заметим, что для первых step-ов все stage будут влезать в локальную память, поэтому вынесем в отдельное ядро функцию, отвечающую за первые step-ы:


```

11 //__kernel void bitonic_sort_gpu_small_blocks_sizes_in_local_mem(__global TYPE* data)
12 {
13     /* assert( data.size >= LOCAL_SIZE) */
14     uint it1 = get_local_id(0);
15     uint git1 = get_global_id(0);
16     uint git2;
17     uint it2;
18     uint block_size = 2;
19     uint stage_comparing_distance;
20
21     __local TYPE ldata[LOCAL_SIZE];
22
23     ldata[it1] = data[git1];
24     barrier(CLK_LOCAL_MEM_FENCE);
25
26     /* small stages */
27     for (/* block_size = 2 */; block_size <= LOCAL_SIZE; block_size <== 1)
28     {
29         for (stage_comparing_distance = (block_size >> 1); stage_comparing_distance > 0; stage_comparing_distance >= 1)
30         {
31             /*
32              * add 2^stage_comparing_distance by module 2^(stage_comparing_distance+1)
33              * thats mean get index of pair in current block, because current_block_size = 2^(stage_comparing_distance+1)
34              * current_block_size != block_size
35              */
36             it2 = it1 ^ stage_comparing_distance;
37             if (it1 < it2)
38             {
39                 TYPE a = ldata[it1];
40                 TYPE b = ldata[it2];
41
42                 if ((!(git1 & block_size)) == (a > b))
43                 {
44                     ldata[it1] = b;
45                     ldata[it2] = a;
46                 }
47             }
48             barrier(CLK_LOCAL_MEM_FENCE);
49         }
50     }
51
52     data[git1] = ldata[it1];
53 }

```

Шире шаг!

Теперь давайте заметим, что даже на больших step-ах у нас есть stage-и в которых все боксы помещаются в локальную память. Поэтому разделим большие step-ы на 2 этапа:

1. выполняем большие stage в глобальной памяти
2. выполняем маленькие stage в локальной памяти

Итерацию же по step и stage мы вынуждены вынести на cpi, так как барьеры могут быть только по локальной памяти (особенность gru), а нам нужна синхронизация после каждого step на всем массиве.

Шаг широкий, глобально

```
55 // __kernel void bitonic_sort_gpu_big_steps_big_stages(__global TYPE* data, uint block_size, uint stage_comparing_di
56 {
57     uint git1 = get_global_id(0);
58     uint git2 = git1 ^ stage_comparing_distance;
59
60     if (git1 < git2)
61     {
62         TYPE a = data[git1];
63         TYPE b = data[git2];
64
65         if ((!(git1 & block_size)) == (a > b))
66         {
67             data[git1] = b;
68             data[git2] = a;
69         }
70     }
71 }
```

Шаг широкий, локально

```
73 // __kernel void bitonic_sort_gpu_big_steps_small_stages(__global TYPE* data, uint block_size)
74 {
75     uint it1 = get_local_id(0);
76     uint git1 = get_global_id(0);
77     uint git2;
78     uint it2;
79     uint stage_comparing_distance;
80
81     __local TYPE ldata[LOCAL_SIZE];
82
83     ldata[it1] = data[git1];
84     barrier(CLK_LOCAL_MEM_FENCE);
85
86     for (stage_comparing_distance = LOCAL_SIZE >> 1 ; stage_comparing_distance > 0; stage_comparing_distance >>= 1)
87     {
88         /*
89          * add 2^stage_comparing_distance by module 2^(stage_comparing_distance+1)
90          * thats mean get index of pair in current block, because current_block_size = 2^(stage_comparing_distance+1)
91          * current_block_size != block_size
92          */
93         it2 = it1 ^ stage_comparing_distance;
94         if (it1 < it2)
95         {
96             TYPE a = ldata[it1];
97             TYPE b = ldata[it2];
98
99             if ((!(git1 & block_size)) == (a > b))
100             {
101                 ldata[it1] = b;
102                 ldata[it2] = a;
103             }
104         }
105         barrier(CLK_LOCAL_MEM_FENCE);
106     }
107
108     data[git1] = ldata[it1];
109 }
```

Дошагали до генштаба

```
254 template <BitonicSortIteratorConcept It>
255 void OpenCLSorting::sort(It begin, It end)
256 {
257     using type = typename It::value_type;
258
259     add_type_define_in_kernel<It>();
260
261     size_t cl_buf_size;
262     cl::Buffer cl_data = copy_input_on_queue(begin, end, &cl_buf_size);
263     msg_assert(cl_buf_size >= LOCAL_SIZE, "this need for simplify of work with small sizes");
264
265     small_blocks_sizes_t bitonic_sort_gpu_small_blocks_sizes_in_local_mem = get_small_blocks_sizes();
266     big_compare_distance_t bitonic_sort_gpu_big_steps_big_stages = get_big_compare_distance();
267     small_compare_distance_t bitonic_sort_gpu_big_steps_small_stages = get_small_compare_distance();
268
269     cl::EnqueueArgs Args1(&queue: queue_, global: cl::NDRange(size0: cl_buf_size), local: cl::NDRange(size0: local_size_));
270     cl::EnqueueArgs Args2(&queue: queue_, global: cl::NDRange(size0: cl_buf_size));
271     cl::EnqueueArgs Args3(&queue: queue_, global: cl::NDRange(size0: cl_buf_size), local: cl::NDRange(size0: local_size_));
272
273     cl::Event Evt = bitonic_sort_gpu_small_blocks_sizes_in_local_mem(args: Args1, cl_data);
274     Evt.wait();
275
276     for (cl_uint block_size = local_size_ << 1; block_size <= cl_buf_size; block_size <= 1)
277     {
278         for (cl_uint stage_comparing_distance = (block_size >> 1); stage_comparing_distance >= local_size_; stage_comparing_distance >= 1)
279         {
280             Evt = bitonic_sort_gpu_big_steps_big_stages(args: Args2, cl_data, block_size, stage_comparing_distance);
281             Evt.wait();
282         }
283
284         Evt = bitonic_sort_gpu_big_steps_small_stages(args: Args3, cl_data, block_size);
285         Evt.wait();
286     }
287
288     cl::copy(queue_, cl_data, begin, end);
289 }
```

И чего шагали, спрашивается (aka бенчмарк)

Что за видеокарта, что за процессор?

Все бенчмарки проводились на моем старичке **Lenovo ThinkPad X1 Carbon Gen 9**, 21 года рождения,

с процессором **11th Gen Intel® Core™ i7-1185G7 × 8**,

и встроенной видеокартой **Intel Iris XE Graphics**

танки еле тянет на минимальных настройках, поэтому достигнутые результаты, о которых далее, должны быть подвергнуты удвоенному восхищению

Посмотрим результаты бенчмарка (все время в миллисекундах):

Elements: 10	Elements: 100	Elements: 1000
Bitonic:	Bitonic:	Bitonic:
19 18 18 18 20 18 19 18 18	19 18 19 18 18 18 19 18 18	19 18 19 18 19 18 18 18 18
std::sort:	std::sort:	std::sort:
0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0
Elements: 10000	Elements: 100000	Elements: 200000
Bitonic:	Bitonic:	Bitonic:
18 18 19 18 18 18 19 18 18	20 18 19 18 20 18 19 18 19	21 23 21 20 23 21 19 19 20
std::sort:	std::sort:	std::sort:
0 0 0 0 0 0 0 0 0	4 4 4 4 4 4 4 4 4	9 9 9 10 9 9 9 10 9
Elements: 300000	Elements: 400000	Elements: 500000
Bitonic:	Bitonic:	Bitonic:
22 23 23 22 21 22 22 32 24	22 23 22 23 27 22 23 24 24	28 24 27 28 22 22 22 23 23
std::sort:	std::sort:	std::sort:
15 15 14 16 14 15 14 14 15	21 21 20 20 19 21 21 20 21	27 28 25 27 25 27 25 26 26

На маленьких массивах, как это и ожидалось, накладные расходы от копирования данных на видеокарту и прочей работы с видеокартой перевешивают эффективность параллельного выполнения сортировки, и потому std::sort с большим отрывом побеждает. Однако...

Elements: 600000	Elements: 700000	Elements: 800000
Bitonic:	Bitonic:	Bitonic:
24 25 26 26 27 32 28 33 26	26 25 26 27 27 29 26 29 28	25 26 26 26 27 26 24 25 25
std::sort:	std::sort:	std::sort:
40 33 33 33 33 32 33 29 31	36 39 37 37 37 35 38 36 35	43 41 43 42 40 40 40 41 43
Elements: 900000	Elements: 1000000	Elements: 10000000
Bitonic:	Bitonic:	Bitonic:
26 26 25 26 26 26 24 26 25	25 26 26 25 25 25 25 25 25	374 382 383 384 386 394 386 388
std::sort:	std::sort:	std::sort:
48 47 47 46 48 48 49 46 46	53 51 52 54 52 53 52 53 51	150 166 150 167 149 163 152 152
Elements: 100000000		
Bitonic:		
1390 1419 1455 1512 1396 1414 1431 1478 1471		
std::sort:		
7864 7065 7943 7858 7872 7974 7967 7037 7892		

... когда мы приближаемся к полумиллиону элементов, преимущество std::sort начинает невеликоваться, а начиная с 600_000 элементов bitonicsort начинает обгонять std::sort, доводя свое преимущество до 5-кратного на 100_000_000 элементов:

ну все, покедова